

SOFIAA OS

Sistema de Inteligencia Operativa Autónoma

Análisis Doctoral del Estado del Sistema

24 de Julio, 2026

[Next.js 16.2.9] [React 19.2.4] [Firebase 12] [Groq LLM] [Vercel]

[9 Extensiones] [4 CRONs] [322+ Sprints] [Producción]

Clasificación: Arquitectura Interna Confidencial

1. Resumen Ejecutivo

SOFIAA OS es un sistema de inteligencia operativa autónoma construido sobre Next.js 16, React 19, Firebase y modelos LLM de Groq. Al 24 de julio de 2026, el sistema se encuentra en producción en Vercel con 9 extensiones funcionales, 4 CRON jobs autónomos, un pipeline cognitivo de 17 pasos y más de 322 sprints completados desde su concepción.

El sistema no es un chatbot: es una plataforma de IA cognitiva que adapta su comportamiento, construye un modelo mental del usuario sesión a sesión, gestiona memoria a largo plazo distribuida, y ejecuta acciones autónomas en sistemas externos vía HERMES.

Versión del sistema	SOFIAA OS v0.1.0 (producción)
Framework	Next.js 16.2.9 con App Router
Runtime JS	React 19.2.4
Base de datos	Firebase Firestore (cliente ^12.15.0 + admin ^14.0.0)
LLM primario	Groq (llama-3.3-70b-versatile / mixtral)
LLM secundario	Google Gemini (@google/generative-ai ^0.24.1)
Deployment	Vercel (Edge + Node.js runtime)
Extensiones activas	9 módulos en ExtensionRegistry
CRON jobs	4 automatizaciones diarias
Sprints completados	322+ (desde Sprint 0 hasta AJ-5)

Premisa doctoral: SOFIAA OS implementa el primer sistema de IA cognitiva multi-extensión para PyMEs y corporativos mexicanos, con memoria semántica distribuida, perfil cognitivo por usuario, gobernanza declarativa y ejecución autónoma de acciones.

2. Filosofía y Visión del Sistema

SOFIAA OS nació como un asistente para PASCALL, la agencia creativa fundada por Abrahan. Con el tiempo evolucionó de chatbot a sistema operativo cognitivo: una plataforma que no solo responde, sino que observa, aprende, actúa y reflexiona sobre sí misma.

2.1 Los Tres Principios Fundacionales

- ▶ URL como señal de activación: la URL del usuario determina qué extensión activa SOFIAA. No hay if/else hardcodeados — el ExtensionRegistry es agnóstico y extensible.
- ▶ Cero latencia en el camino crítico: todas las operaciones de aprendizaje (CognitiveProfile, N.E.X.O. attention, Pipeline Observer) son post-stream y async — nunca bloquean la respuesta.
- ▶ Zero-failure cognitive: el sistema cognitivo nunca puede romper el flujo de chat. Cada capa de N.O.R.A. y N.E.X.O. tiene try/catch silencioso y fallback.

2.2 La Metáfora del Sistema Operativo

El nombre 'OS' (Operating System) es intencional. SOFIAA OS funciona como un sistema operativo para inteligencia artificial:

- ▶ Kernel: el pipeline de chat (route.ts) es el kernel — procesa cada 'syscall' (mensaje del usuario)
- ▶ Drivers: las extensiones son drivers — traducen el dominio específico al lenguaje del sistema
- ▶ File System: N.E.X.O. + Long-term Memory son el sistema de archivos — persistencia jerárquica
- ▶ Scheduler: los 4 CRON jobs son el scheduler — tareas autónomas sin intervención del usuario
- ▶ Process Monitor: N.O.R.A. es el monitor de procesos — observa el comportamiento del sistema
- ▶ Package Manager: ALEJANDRÍA es la memoria de instalación — historia de cada decisión

2.3 Clientes Actuales del Sistema

PASCALL	Agencia creativa — asistente central en /
Tecnológico de Monterrey	Producción audiovisual — TEC BI + TEC BII
Jardines de Juan Pablo	Memorial y servicios fúnebres — JP Memorial
Clientes Marketing Sofia	Clientes de la agencia SMM de PASCALL
ATENA (TEC)	Proyectos Six Sigma / DMAIC de mejora continua

3. Stack Tecnológico

SOFIAA OS utiliza un stack moderno orientado a rendimiento y escalabilidad serverless. Cada dependencia fue elegida con criterios específicos de latencia, costo y capacidad.

3.1 Dependencias de Producción

next@16.2.9	Framework core — App Router, Server Actions, Edge Runtime
react@19.2.4	UI — últimas APIs concurrent, Server Components
firebase@12.15.0	SDK cliente — Firestore, Auth, Storage
firebase-admin@14.0.0	SDK servidor — Admin SDK para CRONs y APIs
@google/generative-ai@^0.24.1	LLM secundario — Gemini para embeddings y análisis
@react-three/fiber@^9.6.1	Neural Orb — 3D canvas en WebGL
@react-three/postprocessing@^3.0.4	Efectos bloom/aurora en el Orb
three@^0.184.0	Three.js — motor 3D del Orb
recharts@^3.8.1	Gráficas — ATENA SPC, NEXO decay, dashboards
jspdf@^4.2.1 + jspdf-autotable@^5.0.8	Exportación PDF — TEC BI + ATENA
pdf-parse@^1.1.4	Extracción de texto de PDFs — N.E.X.O. Biblioteca
react-markdown@^10.1.0 + remark-gfm@^4.0.1	Renderizado Markdown en chat

3.2 Dependencias de Desarrollo

tailwindcss@^4	CSS utility-first — sin custom CSS salvo variables
typescript@^5	Tipado estático end-to-end
vitest@^2	Testing unitario — test suite del Core
@vitest/coverage-v8	Cobertura de código
eslint-config-next@16.2.9	Linting con reglas Next.js

3.3 Proveedores LLM

El sistema implementa un LLM Orchestrator que selecciona el proveedor óptimo por tipo de tarea:

Groq	Primario. TaskTypes: query, extraction. Sub-800ms. Modelos: llama-3.3-70b, mixtral-8x7b
Gemini	Secundario/fallback. TaskTypes: analysis, generation, automation. Capacidad multimodal y contexto amplio
Routing por TaskType	5 tipos: query→Groq, extraction→Groq, analysis→Gemini, generation→Gemini, automation→Gemini

4. Arquitectura Core: El Pipeline de Chat

El corazón de SOFIAA OS es el handler POST de `/api/chat/route.ts`. Es el único punto de entrada para toda conversación y ejecuta un pipeline determinístico de 17+ pasos antes de generar la respuesta.

4.1 El Pipeline Completo (route.ts)

```
POST /api/chat
├── [1] createTracer()                - traceId único por request
├── [2] createEventBus()              - bus asíncrono waitUntil
├── [3] Deserializar body             - messages, userId, activePath,
agentMode...
├── [4] GUARDRAILS servidor           - analyzeMessage() +
analyzeConversation()
│   ├── si violación → getSafetyResponse() → return (sin LLM)
│   ├── extensionRegistry.resolve()  - URL → SofiaaExtension activa
│   ├── getGoalContext()             - estado del Goal Engine
│   ├── getPoliciesForContext()       - políticas cognitivas por scope
│   ├── buildPolicyBlock()            - inyectar políticas al system prompt
│   ├── capabilityGateway.check()     - RBAC antes de datos
│   ├── capabilityRuntime.summarize() - capabilities del usuario
│   ├── getSemanticNexoContext()      - top 5 nodos N.E.X.O. híbridos
│   ├── getBibliotecaContext()        - nodos de PDFs de la biblioteca
│   ├── isAlejandriaQuery() ?        - ¿pregunta sobre arquitectura?
│   │   └── getAlejandriaContext()    - nodos históricos de ALEJANDRÍA
│   ├── getCognitiveProfile()         - perfil cognitivo del usuario
│   ├── buildCognitiveBlock()         - bloque para system prompt
│   ├── buildGraphBlockFromPayload()  - ExperienceGraph del usuario
│   └── assemblePrompt()              - system prompt modular completo
├── [AGT] agentMode ?                - ¿modo agente activado?
│   └── AgentRuntime.run()            - loop ReAct con tools
├── [LLM] orchestrator.stream()       - Groq o Gemini según TaskType
│   └── parseToolCalls()              - Function Calling (nav + UI)
├── STREAM → cliente
└── POST-STREAM (waitUntil - async, no bloquea):
    ├── extractSignals()              - señales cognitivas del usuario
    ├── updateCognitiveProfile()       - actualizar perfil cognitivo
    ├── attendNexoNodes()              - reforzar nodos N.E.X.O. usados
    ├── reinforceAlejandriaNodes()    - reforzar nodos ALEJANDRÍA usados
    ├── evaluateResponse()             - PolicyEvaluator post-stream
    └── bus.emit()                     - eventos al EventBus
```

4.2 Extension Registry v2 (Sprint C1)

El ExtensionRegistry es el mecanismo de activación de extensiones. Opera exclusivamente por URL prefix — el Core nunca sabe qué extensión está activa.

Resolución	pathname.startsWith(routePrefix) → candidatos → selectVersion()
Versioning	Semver. Sin pin → versión más alta. pinVersion(id, v) para staging/rollback
Seguridad	checkAccess(pathname, userRole) — RBAC por extensión
Introspección	listIds(), listVersions(id) — para panel de admin

4.3 Guardrails Doble Capa (Sprint 2.2 / B2)

- ▶ Capa cliente: validación en el frontend antes de enviar al servidor
- ▶ Capa servidor: `analyzeMessage()` + `analyzeConversation()` en `route.ts`
- ▶ Si violación → `getSafetyResponse()` retorna respuesta segura sin invocar LLM
- ▶ Edge Middleware (B2): RBAC antes de llegar al handler — bloqueo en el edge

4.4 Structured Outputs: Function Calling (Sprint B4)

SOFIAA utiliza Function Calling para acciones estructuradas que el LLM debe ejecutar, eliminando el parsing frágil de texto:

<code>[NAVIGATE:path]</code>	Navegar a una ruta de la app
<code>[UI:component:data]</code>	Renderizar componente de UI generativa
<code>SOFIAA_TOOLS</code>	Conjunto de herramientas disponibles para el LLM
<code>parseToolCalls()</code>	Parser del stream para detectar tool calls
<code>serializeToolResults()</code>	Serializar resultados de tools al contexto

5. Ecosistema de Extensiones — 9 Módulos Activos

Al 24 de julio de 2026, SOFIAA OS tiene 9 extensiones registradas en el ExtensionRegistry. Cada una representa un dominio de negocio independiente con su propio manifest, herramientas, UI y colección Firestore.

TEC BI — v1 (Legacy)

Ruta	/tec-bi
Sprint origen	Sprint 1–7, Q
Descripción	Módulo original TEC Monterrey: CRUD de empleados, proveedores, clientes, briefs, proyectos, evaluaciones
Estado	Activo — legacy, convive con TEC BII v2
Color	#0D9488 — Teal

TEC BII — v2 Cognitivo

Ruta	/tec-bii
Sprint origen	T2-0 a T2-6, M-7, Q-1 a Q-3, P (predictivo)
Descripción	Inteligencia cognitiva sobre producción audiovisual TEC. CognitiveFootprint en cada entidad, N.O.R.A. reflection, cross-domain NEXO↔TEC, brief generativo, riesgo predictivo
Extension Tools	5: analizar_briefs, consultar_proyectos, evaluar_equipo, generar_insight, calcular_riesgo
CRONs	/api/tec-bii/refine — diario 11AM UTC (3 fases: republish + N.O.R.A. + cross-domain)
Color	#0D9488 — Teal

N.E.X.O. — Experience Graph

Ruta	/nexo
Sprint origen	N-0 a N-7, M-0 a M-4B
Descripción	Grafo de conocimiento personal del usuario. Ingesta desde Chrome Extension, PWA Share, PDF. Decay exponencial por categoría. Reflexión diaria con Groq. Embeddings híbridos (coseno + weight).
Colecciones	nexo_nodes, nexo_events, biblioteca
CRONs	/api/nexo/decay (9AM UTC) + /api/nexo/reflect (10AM UTC)
Color	#4F46E5 — Indigo

PROMETEO — Growth Intelligence v2

Ruta	/prometeo
Sprint origen	P-0 a P-8
Descripción	Intelligence engine de crecimiento de marca. Brand DNA → N.E.X.O. publish. Goals multi-estado. Creative Memory de performance. Creative Lab (variantes con Groq). Director Autónomo (brief diario). HERMES bridge.
Extension Tools	6: analizar_brand_dna, listar_objetivos, creative_memory, generar_variante, director_brief, publicar_nexo
CRONs	/api/cron/prometeo-brief — diario 8AM UTC
Color	#EA580C — Fire Orange

ATENA — Scientific Intelligence Engine

Ruta	/atena
Sprint origen	A-0 a A-9

Descripción	Motor científico Six Sigma / DMAIC. ANOVA, regresión, cartas de control X-R con alertas Western Electric, Matriz AMEF (NPR), VAN/TIR, Monte Carlo P10/P50/P90, wizard de proyectos, HERMES bridge para NPR crítico.
Extension Tools	5: consultar_proyecto_dmaic, analizar_anova, revisar_amef, calcular_financiero, detectar_alertas_spc
Color	#0891B2 — Cyan

HERMES — Action Execution Layer v1

Ruta	/hermes
Sprint origen	H-0 a H-8
Descripción	Capa de ejecución de acciones en sistemas externos. Cola de acciones pendientes/aprobadas/ejecutadas. Conectores Etapa 1: Monday.com, Slack, Calendario SMM, Interno. PROMETEO bridge + ATENA bridge.
Flujo	PROMETEO/ATENA → encolado → aprobación humana → ejecución → historial
Color	#B45309 — Amber

JP Memorial — Jardines de Juan Pablo

Ruta	/jp-memorial
Sprint origen	Sprint 41–44
Descripción	Asistente empático para servicios fúnebres. Chat flotante JpmChat. Política JP_MEMORIAL_EMPTHY activa (severity=error). Tono de duelo detectado y forzado.
Color	#DB2777 — Rosa

Marketing Sofia — SMM Intelligence

Ruta	/marketing-sofia
Sprint origen	Sprint 46–47, P-8
Descripción	Gestión de clientes SMM, métricas de campaña, calendario editorial, finanzas. Publisher a N.E.X.O. de métricas. Auto-registro en Creative Memory de PROMETEO al cerrar métricas.
Color	#DB2777 — Rosa

ALEJANDRÍA — Memoria Histórica de Ingeniería

Ruta	/alejandria
Sprint origen	AJ-0 a AJ-5
Descripción	Knowledge Graph de la historia de ingeniería de SOFIAA OS. 7 tipos de nodos: sprint, decision_arquitectura, brainstorming, especificacion_modulo, experimento, hito, idea. Activa con triggers de introspección.
Activación	isAlejandriaQuery(): 30+ triggers como 'cómo funciona', 'arquitectura', nombre de módulos
Color	#D97706 — Gold

6. Sistemas de Memoria — 4 Capas

SOFIAA OS implementa cuatro capas de memoria complementarias, cada una con distinto alcance temporal, granularidad y propósito. Juntas forman el sistema de memoria más sofisticado implementado en el ecosistema.

6.1 Long-term Memory (Sprint H-2)

Memoria textual a largo plazo del usuario. Patrón dual-write localStorage + Firestore.

Ruta Firestore	users/{uid}/memory/long_term — campo: content
Patrón	Dual-write: localStorage (zero latency) + Firestore (multi-device)
Sincronización	Al login: syncMemoryFromFirestore(). Firestore gana en conflicto.
API	writeLongMemory, readLongMemory, appendLongMemory, clearLongMemory

6.2 N.E.X.O. — Knowledge Graph Semántico (Sprints N + M)

Grafo de nodos de conocimiento del usuario con decay exponencial, embeddings híbridos y reflexión diaria.

Nodos	users/{uid}/nexo_nodes — NexoNode con 9 categorías, weight, decayRate
Decay	$\lambda = \ln(2)/\text{halfLifeDays}$ — aplicado cada 24h (9AM UTC)
Embeddings	hybridScore = cosine×0.65 + weight×0.35 (EMBEDDING_DIMS=3072, actual: null→weight-only)
Ingesta	Chrome Extension, PWA Share, PDF Library, manual, tec_bii, prometeo, marketing
Reflexión	Diaria 10AM UTC — top 30 nodos → Groq → 3 insights tipo 'conexión_temática'
Atención	Post-stream: attendNexoNodes() refuerza +0.03 los nodos usados en contexto

6.3 Biblioteca PDF (Sprint M-1)

Sub-capa de N.E.X.O. para documentos PDF. Procesamiento con pdf-parse + Groq, máximo 7 nodos por documento.

Colección	users/{uid}/biblioteca — BibliotecaDoc
Ruta API	POST /api/nexo/pdf (10MB max)
Proceso	pdf-parse → texto → Groq 2048 tokens → máx 7 NexoNodes (source: pdf_library, weight=0.75)
Cascade delete	deleteBibliotecaDoc elimina el doc + todos sus nodos asociados

6.4 ALEJANDRÍA — Memoria Histórica (Sprint AJ)

Knowledge Graph de la historia de ingeniería del propio SOFIAA OS.

Colección	alejandria_nodes — AlejandriaNode con 7 tipos
Activación	isAlejandriaQuery(): 30+ triggers semánticos
Ingestión	Script alejandria:ingest — corpus JSON histórico → Firestore
Refuerzo	reinforceAlejandriaNodes() post-stream — misma lógica que N.E.X.O.

Principio arquitectónico: Las 4 capas de memoria operan independientemente pero se alimentan mutuamente: PROMETEO publica a N.E.X.O., Marketing Sofia publica a N.E.X.O., TEC BII publica a N.E.X.O. ALEJANDRÍA es de sólo lectura — la historia no se modifica, solo se consulta.

7. Capa Cognitiva — N.O.R.A. + CognitiveProfile + Policies

La capa cognitiva es lo que distingue a SOFIAA OS de un chatbot convencional. Opera en tres dimensiones: el perfil del usuario (CognitiveProfile), la gobernanza del comportamiento (CognitivePolicy) y la observabilidad del sistema (N.O.R.A.).

7.1 CognitiveProfile (Sprint M-3)

Modelo mental persistente del usuario. Almacenado en `users/{uid}/cognitive_profile/v1`. Evoluciona cada sesión.

preferredDepth	'concise' 'balanced' 'deep' — transición gradual por señales
depthConfidence	0–1 — confianza en la preferencia. Solo se aplica si >0.15
formalityScore	0.0–1.0 — 0=casual, 0.5=neutral, 1.0=formal. Delta: ±0.08 por señal
topicAffinity	Record<string, number> — 8 temas, acumula +0.12/señal, decay 0.5%/sesión
sessionCount	Contador de sesiones — proxy de antigüedad del usuario

7.2 Signal Extractor (signals.ts)

Extracción de señales cognitivas por regex — sin LLM, sin latencia, sin fallo posible.

depth_increase	17 patrones (más detalle, profundiza, amplía...) — confianza 0.85
depth_decrease	15 patrones (al grano, en resumen, brevemente...) — confianza 0.85
formality_up	11 patrones (usted, estimado, le solicito...) — confianza 0.70
formality_down	16 patrones (wey, jaja, bro, xd...) — confianza 0.70
topic_mention	8 temas × n keywords — confianza: min(1, matches×0.30)

7.3 CognitivePolicy Engine (Sprint D-D)

Sistema declarativo de gobernanza. 8 políticas en el registry, evaluadas post-stream.

POLICY_TONE_SOFIAA	global, priority=100 — no_robotic, no_apologies, spanish_first
POLICY_PRIVACY	global, priority=95 — no_expose_keys (sk-*, Bearer, Alza*), no_expose_paths
POLICY_NAVIGATION	global, priority=90 — exact_routes (max 1 NAVIGATE token)
POLICY_JP_MEMORIAL	extension, priority=85 — empathy_first (severity=error), no_price_pressure
POLICY_GOAL_ACTIVE	goal_active, priority=92 — one_question_per_step, confirm_step_data
POLICY_UNAUTHENTICATED	unauthenticated, priority=88 — gentle_auth_prompt
POLICY_TEC_BI	extension, priority=80 — formal_language, data_grounded
POLICY_MARKETING_SOFIA	extension, priority=80 — actionable_ideas

7.4 N.O.R.A. Pipeline Observer (Sprints F-3 / H-3)

Telemetría de grano fino. Cada request genera un PipelineEvent con dual-write localStorage + Firestore.

Evento	id, timestamp, messageSnippet, taskType, provider, confidence, cacheLayer, latencyMs
CacheLayer	exact semantic miss — KPI de eficiencia del pipeline
Firestore	users/{uid}/pipeline_events/{id} + campo date para queries
Stats	cacheHitRate, avgLatencyMs, taskDistribution, providerDistribution
NoraPanel	Panel admin con 3 pestañas (Usuarios, Eventos, Distribución) — activado por 'Fiat lux'

8. Inteligencia Autónoma — Agent Runtime y HERMES

SOFIAA OS supera el modelo request-response estático con dos capas de autonomía: el Agent Runtime para razonamiento multi-paso dentro de una conversación, y HERMES para ejecución de acciones en sistemas externos.

8.1 Agent Runtime — Loop ReAct (Sprints G-1/G-2/G-3)

Cuando agentMode=true, SOFIAA utiliza el paradigma Reasoning + Acting para descomponer tareas complejas en pasos.

```
AgentRuntime.run(context)
while (steps < MAX_STEPS):
  1. Think: ¿qué necesito saber para responder?
  2. Act:   invocar tool (query_data | search_memory | analyze)
  3. Observe: procesar resultado del tool
  4. Repeat o → Final Answer
```

query_data	Consulta datos estructurados de Firestore (extensiones, colecciones)
search_memory	Busca en Long-term Memory + nodos N.E.X.O. del usuario
analyze	Llama a /api/analyze — análisis Groq especializado sin contaminar el chat
/api/analyze	POST {data, instruction} → Groq 512 tokens, temp=0.3, max 300 palabras

8.2 HERMES — Action Execution Layer (Sprints H-0 a H-8)

HERMES es la capa de ejecución de acciones en sistemas externos, con aprobación humana antes de ejecutar.

Flujo	Encolado → Aprobación manual → Ejecución → Historial
Conectores v1	Monday.com, Slack, Calendario SMM, Interno
PROMETEO bridge	DirectorBrief generado → HermesActions encoladas automáticamente
ATENA bridge	NPR crítico detectado → alerta HERMES encolada
Colección	users/{uid}/hermes_actions + hermes_historial

8.3 Goal Engine (Sprint D-A)

Sistema de objetivos multi-paso que guía conversaciones con propósito definido.

GoalType	6 tipos de objetivo detectados en el mensaje del usuario
GoalState	Máquina de estados: idle → active → collecting → executing → done
buildGoalPromptBlock	Genera instrucciones de un paso por vez — POLICY_GOAL_ACTIVE enforced
Persistencia	Estado del goal en el payload del request (client-managed)

8.4 Capability Runtime (Sprint E)

Capa de acceso a datos en tiempo real bajo demanda, con RBAC y auditoría.

capabilityGateway	Verifica RBAC antes de exponer datos al LLM
capabilityRuntime	Registra providers (Firestore o Mock)
firestoreProvider	Lee colecciones de extensiones autorizadas para el rol activo
buildCapabilityMenuBlock	Bloque del system prompt: qué datos puede pedir el LLM

9. El Sistema Nervioso Autónomo — 4 CRON Jobs

SOFIAA OS opera tareas autónomas diarias sin intervención del usuario. Los 4 CRON jobs registrados en vercel.json forman el 'sistema nervioso autónomo' del sistema: refinan, decaen, reflexionan y actúan por cuenta propia.

08:00 UTC	/api/cron/prometeo-brief — PROMETEO Director Brief diario (3AM CST)
09:00 UTC	/api/nexo/decay — N.E.X.O. Decay Engine (4AM CST)
10:00 UTC	/api/nexo/reflect — N.E.X.O. Reflection Engine (5AM CST)
11:00 UTC	/api/tec-bii/refine — TEC BII Batch Refinement (6AM CST)

9.1 PROMETEO Director Brief (08:00 UTC)

El CRON más temprano genera el DirectorBrief diario de PROMETEO y encola las acciones resultantes en HERMES.

- ▶ Lee Brand DNA + Creative Memory + Goals del usuario
- ▶ Groq genera un DirectorBrief estructurado (estrategia del día)
- ▶ prometeo-bridge.ts convierte el brief en HermesActions encoladas
- ▶ Director queda listo para aprobación humana antes de ejecutar

9.2 N.E.X.O. Decay (09:00 UTC)

Aplica la función de decaimiento exponencial a todos los nodos de todos los usuarios.

- ▶ collectionGroup discovery: encuentra todos los nexo_nodes activos en Firestore
- ▶ $\lambda = \ln(2)/\text{halfLifeDays}$ — decay adaptativo por categoría (food=14d, brand_identity=90d)
- ▶ Nodos con weight < 0.05 (NEXO_PRUNE_THRESHOLD) son eliminados
- ▶ Procesamiento en batch — una sola pasada por usuario

9.3 N.E.X.O. Reflection (10:00 UTC)

Genera insights sintéticos de segundo orden a partir de los nodos capturados del usuario.

- ▶ Cooldown 20h — máximo ~1 reflexión/día por usuario
- ▶ Top 30 nodos captured con weight > 0.1
- ▶ Groq temp=0.75 — genera máx 3 insights tipo 'conexión_temática', 'curiosidad', 'hábito'
- ▶ Insight nodes: decayRate=0.02 (decaen muy lento), weight=0.85

9.4 TEC BII Refine (11:00 UTC)

El más complejo: 3 fases secuenciales por usuario para mantener el grafo cognitivo TEC BII actualizado.

- ▶ Fase 1: republishStaleEntities — nexoNodeId vacío O lastCognitiveSync > 24h → republicar
- ▶ Fase 2: runNoraReflection — N.O.R.A. reflexión institucional (cooldown 6h, 20 proyectos + 15 empleados + 15 briefs)
- ▶ Fase 3: runCrossDomainAnalysis — hipótesis cruzadas NEXO ↔ TEC BII (threshold 0.60)
- ▶ Persiste RefineResult en tec_bii_refine_logs/{runId}

10. Seguridad y Gobernanza

10.1 Seguridad del Sistema

Variables de entorno	.env.local — gitignored, nunca en repositorio. API keys solo por nano/terminal.
Firebase Auth	UID como identificador primario. Token verificado server-side en cada request protegido
Edge Middleware	RBAC verificado antes del handler (Sprint B2) — bloqueo en el edge de Vercel
CRON Secret	CRON_SECRET en Vercel env — Bearer token para autorizar endpoints /api/*/cron
Guardrails	Doble capa (cliente + servidor) contra mensajes maliciosos
POLICY_PRIVACY	Detecta y bloquea exposición de keys, tokens, paths internos en respuestas

10.2 RBAC — Control de Acceso por Rol

admin	Acceso total — todas las extensiones, NoraPanel, panel de admin
director	Acceso a extensiones de negocio — TEC BII, PROMETEO, ATENA
vp	Vista ejecutiva — dashboards y resúmenes
user	Acceso básico — chat, extensiones públicas
unauthenticated	Solo página principal y JP Memorial — gentle_auth_prompt policy activa

10.3 Trazabilidad

Tracer (Sprint B1)	traceld UUID por request — persiste en Firestore structured logs
EventBus (Sprint B3)	Eventos asíncronos post-stream — no bloquean la respuesta
Pipeline Observer	users/{uid}/pipeline_events — cada request registrado
RefineLog	tec_bii_refine_logs/{runId} — resultado de cada CRON batch
HERMES historial	Trazabilidad completa de acciones ejecutadas en sistemas externos

10.4 CognitivePolicy Engine como Capa de Compliance

El CPE actúa como una capa de compliance automática. Sus detectores post-stream verifican que el LLM haya respetado las instrucciones inyectadas en el system prompt:

- severity='error': violaciones críticas (credenciales expuestas, idioma incorrecto, empatía forzada en jp-memorial)
- severity='warn': violaciones menores (tono robótico, disculpas excesivas)
- Violaciones reportadas al EventBus para logging y análisis

11. Deployment y Observabilidad

11.1 Arquitectura de Deployment (Vercel)

Plataforma	Vercel — Serverless + Edge Functions
API Routes	runtime: 'nodejs' para todas las rutas que usan Firebase Admin o pdf-parse
Streaming	Vercel Edge con ReadableStream para respuestas LLM en tiempo real
maxDuration	60s — APIs pesadas (PDF, CRON, N.O.R.A. reflection)
CRON Jobs	4 jobs en vercel.json — ejecutados por Vercel Cron automáticamente
Environment	.env.local local / Vercel Environment Variables en producción

11.2 Rutas API Activas

POST /api/chat	Handler principal — streaming con LLM
POST /api/analyze	Analyze endpoint — Agent Tool sin recursión
POST/DELETE /api/nexo/pdf	Upload y eliminación de PDFs de biblioteca
GET /api/nexo/decay	CRON decay — 9AM UTC
GET/POST /api/nexo/reflect	CRON + manual reflection — 10AM UTC
GET /api/nexo/ingest	Ingesta de nodos N.E.X.O.
POST /api/tec-bii/insights	Análisis predictivo de insights
POST /api/tec-bii/brief-gen	Generación de briefs con Groq
GET /api/tec-bii/risk	Panel de riesgo predictivo
POST /api/tec-bii/nora-reflect	N.O.R.A. reflexión manual
GET /api/tec-bii/refine	CRON batch refinement — 11AM UTC
POST /api/tec-bii/cross-domain	Análisis de hipótesis cruzadas
POST /api/prometeo/director/brief	Director Brief + encolar en HERMES
GET /api/cron/prometeo-brief	CRON Director Brief diario — 8AM UTC
POST /api/prometeo/creative-lab/generate	Generación de variantes creativas
POST /api/atena/generar-proyecto	Wizard DMAIC — generar proyecto con Groq
POST /api/atena/hermes-scan	ATENA → HERMES bridge de alertas
POST /api/hermes/execute	Ejecutar acción aprobada
GET /api/alejandria/stats	Estadísticas del Knowledge Graph

11.3 Observabilidad (N.O.R.A.)

La observabilidad del sistema está centralizada en N.O.R.A., accesible desde el panel admin con la frase 'Fiat lux':

- ▶ Pestaña Usuarios: roles, conteos, fechas de registro
- ▶ Pestaña Eventos: pipeline_events de todos los usuarios, ordenados por timestamp
- ▶ Pestaña Distribución: histogramas de taskType y cacheLayer
- ▶ KPIs globales: usuarios totales, interacciones, latencia promedio, cache hit rate, usuarios con memoria

11.4 Scripts de Mantenimiento

biblioteca:ingest	Carga masiva de PDFs de la biblioteca a Firestore
nexo:backfill-embeddings	Backfill de embeddings para nodos existentes sin vector
atena:seed	Carga datos DMAIC de simulación para demo TEC
alejandria:ingest	Ingestión del corpus histórico de SOFIAA a Firestore

12. Evolución del Sistema — Historia de Sprints

SOFIAA OS se construyó en más de 322 sprints distribuidos en múltiples fases. La siguiente línea de tiempo muestra la evolución de la arquitectura desde el primer chatbot hasta el sistema cognitivo autónomo actual.

Fase 0 — Chatbot MVP (Sprints 0–5)

- ▶ Primer chatbot con Gemini API y system prompt estático
- ▶ Neural Orb 3D con Three.js y @react-three/fiber
- ▶ Personalidad JARVIS, tema claro Apple, acciones rápidas
- ▶ Panel de admin con frase secreta
- ▶ Deploy inicial en Vercel

Fase 1 — Architecture Freeze (Sprints 2.1–2.8)

- ▶ Guardrails doble capa (Sprint 2.2)
- ▶ Semantic Cache con normalización de intención (2.4)
- ▶ Goal Engine v1 — detección de objetivos (2.5)
- ▶ Memory Timeline — historial de sesiones (2.6)
- ▶ Visual Intelligence Layer — UI generativa (2.7)

Fase 2 — Extension System v2 (Sprints 1.1.4 A/B/C)

- ▶ Contrato formal SofiaaExtension — extensiones como plugins (A1)
- ▶ ExtensionRegistry agnóstico — URL como señal de activación (A2/A3/A4)
- ▶ Tracer + EventBus + Edge Middleware RBAC (B1/B2/B3)
- ▶ Function Calling para navegación y UI generativa (B4)
- ▶ Semantic versioning + Tool Handler por extensión (C1/C2/C3)

Fase 3 — Cognitive Architecture (Sprint D)

- ▶ LLM Orchestrator con 5 TaskTypes y routing dinámico (D-C)
- ▶ Goal Engine v2 — state machine multi-paso (D-A)
- ▶ Intent Engine + IntentDrivenUI (D-B)
- ▶ CognitivePolicy Engine — gobernanza declarativa (D-D)
- ▶ ExperienceGraph — afinidad por extensión y tema (D-E)

Fase 4 — Capability + Performance (Sprints E/F)

- ▶ Capability Runtime Layer con RBAC y auditoría (E-1 a E-6)
- ▶ Smart Task Classifier 5 tipos (F-1)
- ▶ Semantic Cache con Gemini Embeddings (F-2)
- ▶ Pipeline Observer — telemetría de pipeline (F-3)

Fase 5 — Agent Runtime + Memory (Sprints G/H/I)

- ▶ Agent Runtime ReAct con 3 tools (G-1/G-2/G-3)
- ▶ Long-term Memory dual-write Firestore (H-2)
- ▶ Pipeline Observer extendido a Firestore (H-3)
- ▶ N.O.R.A. Observer Panel — Fiat lux (I-0)

Fase 6 — Branding + N.E.X.O. (Sprints L/N/M)

- ▶ Logo PNG, favicon, Neural Orb Aurora (L1/L2/L3)
- ▶ N.E.X.O. foundation — tipos, Chrome Extension, ingesta LLM (N-0 a N-7)
- ▶ Reflection Engine, Biblioteca PDF, Attention Engine, embeddings híbridos (M-0 a M-4B)
- ▶ CognitiveProfile — perfil mental del usuario (M-3)

Fase 7 — TEC BII Cognitivo (Sprints T2/Q/P predictivo)

- ▶ TEC BII v2 con CognitiveFootprint en cada entidad (T2-0 a T2-6)
- ▶ N.O.R.A. reflection TEC BII, cross-domain NEXO↔TEC BII (T2-4/T2-6)
- ▶ RBAC Context + Extension Tools v2 (Q-1/Q-2/Q-3)
- ▶ Riesgo predictivo — alertas basadas en reglas (Sprint P predictivo)
- ▶ Refine CRON — batch de 3 fases diario (M-7)

Fase 8 — Extensiones Especializadas (Sprints A/P-PROMETEO/H-HERMES)

- ▶ ATENA Six Sigma — ANOVA, SPC, AMEF, Monte Carlo, wizard DMAIC (A-0 a A-9)
- ▶ PROMETEO v2 — Brand DNA, Goals, Creative Lab, Director Autónomo (P-0 a P-8)
- ▶ HERMES v1 — Action Execution, PROMETEO+ATENA bridges, historial (H-0 a H-8)

Fase 9 — ALEJANDRÍA (Sprints AJ)

- ▶ Knowledge Graph histórico de ingeniería de SOFIAA OS (AJ-0 a AJ-5)
- ▶ Corpus JSON ingestado — 7 tipos de nodos, búsqueda semántica
- ▶ Integración en chat/route.ts con 30+ triggers de introspección

13. Estado del Sistema — 24 de Julio, 2026

Snapshot del estado actual del sistema en producción al día de la emisión de este reporte doctoral.

13.1 Componentes en Producción

Core Pipeline	✓ Operativo — 17 pasos, streaming, post-stream async
ExtensionRegistry v2	✓ 9 extensiones registradas, semver activo
N.E.X.O.	✓ Decay + Reflection CRONs diarios operativos
TEC BII	✓ Cognitivo v2 — N.O.R.A., cross-domain, refine CRON
PROMETEO v2	✓ Director Autónomo + CRON diario + HERMES bridge
ATENA	✓ DMAIC + wizard + HERMES bridge para NPR crítico
HERMES v1	✓ Cola + aprobación + ejecución + historial
ALEJANDRÍA	✓ Corpus ingestado, búsqueda semántica activa
N.O.R.A.	✓ NoraPanel activo vía 'Fiat lux', Pipeline Observer Firestore
CognitiveProfile	✓ Actualización post-stream, inyección en system prompt
CognitivePolicy	✓ 8 políticas activas, evaluación post-stream
Agent Runtime	✓ ReAct con 3 tools, /api/analyze independiente
JP Memorial	✓ Chat flotante, política de empatía activa
Marketing Sofia	✓ SMM con publisher a N.E.X.O.

13.2 Arquitectura Firestore — Colecciones Activas

usuarios/{uid}	UsuarioDoc — rol, email, nombre, createdAt
users/{uid}/memory/long_term	Memoria textual del usuario — content, updatedAt
users/{uid}/pipeline_events/{id}	Telemetría N.O.R.A. — un doc por request
users/{uid}/cognitive_profile/v1	CognitiveProfile — perfil cognitivo evolutivo
users/{uid}/nexo_nodes/{id}	Knowledge Graph personal — hasta 9 categorías
users/{uid}/nexo_events/{id}	Log de eventos de ingesta N.E.X.O.
users/{uid}/biblioteca/{docId}	Metadatos de PDFs procesados
users/{uid}/tec_bii_reflexiones/{id}	N.O.R.A. reflexiones institucionales TEC BII
users/{uid}/hermes_actions/{id}	Cola de acciones HERMES
users/{uid}/hermes_historial/{id}	Historial de acciones ejecutadas
tec_bii_refine_logs/{runId}	Resultado de cada CRON batch TEC BII
alejandria_nodes/{id}	Knowledge Graph histórico de ingeniería

13.3 Documentación Doctoral Generada

A la fecha de este reporte, se han generado los siguientes análisis doctorales individuales de cada módulo:

TEC_BII_Reporte_Doctoral.docx	27.8 KB — Sprint T2, M-7, N.O.R.A. TEC BII, Cross-domain
NEXO_Reporte_Doctoral.docx	29.8 KB — Foundation N.E.X.O., Reflection, PDF, Embeddings
NORA_Reporte_Doctoral.docx	25.1 KB — Observer Panel, CognitiveProfile, Policy Engine
PROMETEO_Reporte_Doctoral.docx	generado — Brand DNA, Goals, Creative Lab, Director
ATENA_Reporte_Doctoral.docx	generado — DMAIC, ANOVA, SPC, AMEF, Monte Carlo
HERMES_Reporte_Doctoral.docx	generado — Execution Layer, bridges, conectores
ALEJANDRIA_Reporte_Doctoral.docx	generado — Knowledge Graph, ingesta, búsqueda
SOFIAA_Reporte_Doctoral_2026.docx	ESTE DOCUMENTO — análisis maestro integrado

14. Análisis Sistémico — Patrones Emergentes

Tras el análisis completo del sistema, emergen patrones arquitectónicos transversales que definen la identidad técnica de SOFIAA OS y justifican su clasificación como sistema de inteligencia operativa de nivel doctoral.

14.1 Principio de Composabilidad Cognitiva

Cada módulo de SOFIAA OS emite conocimiento a N.E.X.O., que luego es consultado por todos los demás módulos. Esto crea un loop de retroalimentación cognitiva:

```
PROMETEO → brand_dna → nexo_nodes
TEC BII → proyectos/empleados → nexo_nodes (via cognitive-publisher)
Marketing Sofia → clientes/métricas → nexo_nodes
Chrome Extension / PDF → nexo_nodes

N.E.X.O. → (hybridScore) → system prompt → LLM
LLM → respuesta enriquecida → attend(nodeIds) → weight++
```

Resultado: el sistema aprende de sus propias interacciones y enriquece el contexto de forma acumulativa.

14.2 Principio de Autonomía Progresiva

SOFIAA OS implementa tres niveles crecientes de autonomía:

Nivel 1 — Reactivo	Chat: responde a mensajes del usuario. Latencia <1s.
Nivel 2 — Reflexivo	CRONS: aprende, decae, reflexiona sin intervención del usuario. 4 veces/día.
Nivel 3 — Activo	HERMES: encola y ejecuta acciones en sistemas externos. Con aprobación humana.

14.3 Principio de Seguridad por Diseño

La seguridad no es una capa añadida — es estructural:

- ▶ Zero trust: cada API verifica el token de Firebase. CRON_SECRET para automatizaciones.
- ▶ RBAC en 3 capas: Edge Middleware + capabilityGateway + extensión manifest.security
- ▶ CognitivePolicy como compliance: el LLM no puede exponer credenciales incluso si es instruido
- ▶ Pipeline Observer como audit trail: cada request tiene trazabilidad en Firestore

14.4 Principio de Degradación Elegante

SOFIAA OS está diseñado para nunca fallar visiblemente ante el usuario:

- ▶ getCognitiveProfile(): null silencioso → chat sin perfil
- ▶ getSemanticNexoContext(): error silencioso → sin contexto N.E.X.O.
- ▶ attendNexoNodes(): error silencioso — refuerzo es best-effort
- ▶ Memory Store: localStorage de fallback cuando Firestore falla
- ▶ LLM Orchestrator: Gemini como fallback de Groq en caso de fallo

15. Conclusión Doctoral

Al 24 de julio de 2026, SOFIAA OS representa uno de los sistemas de inteligencia artificial aplicada más sofisticados construidos para el mercado iberoamericano de PyMEs y corporativos. En 322+ sprints, el sistema evolucionó de un chatbot simple a una plataforma cognitiva autónoma con las siguientes características sin precedente en sistemas similares:

15.1 Capacidades Únicas del Sistema

- ▶ 9 extensiones de dominio en un solo sistema de IA — desde memorial fúnebre hasta Six Sigma
- ▶ Pipeline cognitivo de 17 pasos con 4 capas de memoria complementarias
- ▶ Perfil cognitivo por usuario que evoluciona sesión a sesión sin intervención
- ▶ 4 automatizaciones autónomas diarias que refinan el sistema mientras el usuario duerme
- ▶ Ejecución de acciones en sistemas externos con aprobación humana (HERMES)
- ▶ Autoconocimiento: ALEJANDRÍA permite a SOFIAA responder preguntas sobre su propia arquitectura
- ▶ Reflexión institucional: N.O.R.A. genera narrativas del estado de la organización

15.2 Métricas de Complejidad

Colecciones Firestore	12+ colecciones activas en producción
API Endpoints	20+ rutas API con lógica de negocio
CRON Jobs	4 automatizaciones diarias coordinadas
Políticas cognitivas	8 políticas declarativas en el registry
Tipos TypeScript	50+ interfaces y tipos documentados
Extensiones	9 módulos de dominio completamente independientes
Sprints completados	322+ sprints desde el primer commit
Reportes doctorales	8 documentos de análisis técnico profundo

15.3 Visión a Futuro

Los comentarios en el código y la arquitectura del sistema anticipan las siguientes evoluciones:

- ▶ HERMES Etapa 2: conectores de ads (Meta, Google), CRM, facturación
- ▶ N.E.X.O. Embeddings reales: cuando Groq habilite endpoint de embeddings (EMBEDDING_DIMS=3072 listo)
- ▶ CognitiveProfile Fase 2: refinamiento periódico con LLM (no solo regex)
- ▶ ATENA → producción real: integrar datos reales de TEC Monterrey (seed de simulación ya funciona)
- ▶ NEXO Chrome Extension: publicar en Chrome Web Store
- ▶ Multi-tenant: arquitectura multi-organización con namespacing de colecciones

SOFIAA OS

Sistema de Inteligencia Operativa Autónoma

24 de Julio, 2026 · Estado: Producción · Versión: 0.1.0

Clasificación: Arquitectura Interna Confidencial